

PROJETO INCREMENTAL — MARCO 1

CEUB Music — Streaming de Música

Modelagem Inicial de Persistência em Bancos de Dados NoSQL

GRUPO
Centaurus

CAMPUS
Taguatinga

DATA
29/09/2026

DISCIPLINA
NoSQL & Big Data

1. Descrição do Problema

1.1 Contexto da Aplicação

O CEUB Music é uma plataforma de streaming voltada para a comunidade universitária do CEUB, incluindo estudantes, professores e colaboradores. A proposta segue a ideia de serviços como Spotify e Apple Music e, neste primeiro marco, considera quatro operações principais:

1. Sessão de Reprodução Ativa Guarda a faixa atual, a posição em segundos e o estado do player (play/pause). Como são dados temporários, o acesso precisa ser rápido.	2. Histórico de Reproduções Registra as reproduções de cada usuário. Esses dados servem para histórico recente, relatórios e recomendações.
3. Rankings em Tempo Real Mantém listas como “Top Em Alta” e “Top Gênero”, atualizadas conforme novas reproduções acontecem.	4. Gestão de Playlists Permite criar playlists e adicionar, remover ou reordenar músicas com baixa latência.

1.2 Requisitos Principais & Volumetria Estimada

A arquitetura foi dimensionada para a escala operacional do primeiro ano na comunidade acadêmica:

Métrica / Requisito	Estimativa Operacional	Impacto Arquitetural
Base de Usuários Ativos	8.000 usuários cadastrados	Necessidade de isolamento e partição por usuário.
Catálogo de Músicas	500.000 faixas cadastradas	Consultas rápidas e diretas por chave única de faixa.
Reproduções Diárias (Méd / Pico)	40.000 méd. / 80.000 no pico	Picos concentrados nos intervalos de aula e almoço.
Vazão de Escrita (Pico)	~200 escritas simultâneas/s	Inviabiliza locks transacionais em bancos relacionais.
Crescimento do Histórico	~14,6 milhões de logs/ano (~2,9 GB)	Exige banco Wide-Column orientado a append distribuído.

Desafio do Pico de Carga: estima-se que mais de 60% do tráfego diário ocorra em janelas curtas, principalmente entre 9h50–10h05 e 12h00–12h30. O sistema precisa absorver esses picos sem atrasar o player.

2. Modelagem Inicial dos Dados

Para o projeto foram usados dois modelos NoSQL: Redis para dados temporários e operações rápidas em memória, e Cassandra para o histórico persistente de maior volume.

2.1 Dados em Chave-Valor — Redis EM MEMÓRIA

A. Sessão Ativa (Hash + TTL)

Mantém o estado atual do player. O TTL de 2 horas remove sessões que ficaram inativas.

```
KEY: sessao:usuario:5021
HASH fields:
  musica_atual -> "track:88201"
  posicao_seg -> 147
  status -> "playing"
TTL: 7200s
```

B. Fila de Reprodução (List)

A lista guarda as próximas faixas na ordem em que serão reproduzidas.

```
KEY: fila:usuario:5021
LIST items:
  [0] "track:44102"
  [1] "track:77503"
  [2] "track:91204"
```

C. Rankings em Tempo Real (Sorted Set)

A pontuação da faixa aumenta a cada play. ZREVRANGE recupera as mais ouvidas.

```
KEY: ranking:global:diario:2026-09-29
ZSET:
  track:88201 -> score: 1427
  track:44102 -> score: 1183
```

D. Playlists e Contadores (Hash & String)

Hash guarda metadados da playlist e String permite atualizar contadores com INCR.

```
KEY: playlist:usuario:5021:rock
HASH: { nome: "Rock", faixas: 25 }
KEY: plays:track:88201 -> 234567
```

2.2 Dados em Wide-Column — Apache Cassandra QUERY-FIRST

A. Histórico de Reproduções por Usuário

Consulta principal: listar as últimas N músicas ouvidas por um usuário.

```
CREATE TABLE historico_por_usuario (
  usuario_id int,
  reproduzido_em timestamp,
  track_id text,
  nome_musica text,
  artista text,
  duracao_seg int,
  PRIMARY KEY ((usuario_id), reproduzido_em)
) WITH CLUSTERING ORDER BY (reproduzido_em DESC);
```

Partition Key (usuario_id): mantém o histórico de cada usuário na mesma partição.

Clustering Column (reproduzido_em DESC): deixa os registros organizados do mais recente para o mais antigo.

B. Analytics de Reproduções por Artista (Desnormalização Orientada a Consulta)

Uma segunda tabela organiza as reproduções por artista. Assim, essa consulta pode ser feita diretamente, sem depender de JOIN.

```
CREATE TABLE reproducoes_por_artista (
  artista_id text,
  reproduzido_em timestamp,
  track_id text,
  PRIMARY KEY ((artista_id), reproduzido_em, track_id)
) WITH CLUSTERING ORDER BY (reproduzido_em DESC);
```

2.3 Diagrama do Fluxo de Arquitetura de Dados

O fluxo abaixo mostra o que acontece quando o usuário inicia uma reprodução. O Redis recebe as atualizações rápidas e o Cassandra registra o histórico.

CLIENTE / PLAYER WEB & MOBILE	
Evento: Play / Heartbeat	
CAMADA DE APLICAÇÃO	
Escrita em Memória	Escrita Assíncrona
REDIS (Chave-Valor) • Update Hash (Sessão) • ZINCRBY (Ranking Diário) • INCR (Contador Global)	CASSANDRA (Wide-Column) • INSERT historico_por_usuario • INSERT reproducoes_por_artista • Modelagem Query-First

3. Justificativa Tecnológica

3.1 Unidade 1: Fundamentos de Sistemas Distribuídos

Gargalos e Limites do Modelo Relacional

1. Custo de Joins Sob Carga: para montar rankings no SQL seriam necessárias consultas com JOIN, GROUP BY e COUNT, que podem disputar recursos com as gravações em horários de pico.
2. Contenção de Locks em Contadores: muitas atualizações no mesmo contador podem gerar concorrência sobre o mesmo registro.
3. Escala Vertical Limitada: o histórico cresce continuamente e pode exigir aumento constante de recursos em um servidor centralizado.

Teorema CAP: Opção AP

O CEUB Music prioriza Disponibilidade (A) e Tolerância a Partição (P). Uma pequena defasagem no ranking durante uma falha de comunicação é aceitável, desde que a reprodução continue funcionando.

Modelo BASE vs. ACID

A proposta usa BASE nos dados distribuídos: o serviço continua disponível, estados temporários podem expirar e as réplicas podem levar um pequeno intervalo até convergir.

Estratégia de Sharding e Replicação

Tecnologia	Estratégia de Sharding	Estratégia de Replicação
Cassandra	Hash-based pelo Murmur3 Partitioner usando usuario_id como Partition Key. Distribuição uniforme sem hotspots.	Master-Master (Peer-to-Peer): Fator de Replicação RF=3. Sem ponto único de falha; qualquer nó aceita escritas.
Redis	Consistent Hashing via Redis Cluster dividido em 16.384 hash slots. Minimiza re-hashing ao adicionar nós.	Master-Slave + Redis Sentinel: Leituras escaladas nas réplicas e failover automático do Master em caso de queda.

3.2 Unidade 2: Chave-Valor (Redis) & Dynamo Parallel

No CEUB Music, o Redis foi escolhido para os dados que mudam com frequência e precisam de acesso rápido. As estruturas usadas são:

- Hashes para sessão e playlists;
- Lists para a fila de reprodução;
- Sorted Sets para os rankings em tempo real;
- Strings com INCR para contadores;

- TTL para remover estados temporários automaticamente.

O Redis Cluster também distribui as chaves entre nós, o que ajuda a repartir a carga quando o uso aumenta.

3.3 Unidade 3: Wide-Column (Cassandra)

No Cassandra, a modelagem é feita a partir das consultas que precisam ser atendidas. Por isso, informações de uma mesma reprodução podem aparecer em mais de uma tabela, cada uma preparada para um tipo de leitura.

- Column Families Orientadas a Consulta: cada tabela CQL atende a um padrão de acesso específico. A duplicação de alguns dados é intencional para evitar JOINS.
- Partition Key: define em qual partição os registros ficam armazenados, como `usuario_id`.
- Clustering Column: define a ordem dos registros dentro da partição, como `reproduzido_em` DESC.

4. Matriz Resumo da Arquitetura NoSQL

Caso de Uso	Modelo NoSQL	Tecnologia	Mecanismo Estrutura Chave /	Garantia CAP
State do Player	Chave-Valor	Redis	Hash com TTL de 7200 segundos	AP (Soft State)
Fila do Player	Chave-Valor	Redis	List (FIFO com LPUSH / RPOP)	AP (Baixa latência)
Top "Em Alta"	Chave-Valor	Redis	Sorted Set (ZINCRBY / ZREVRANGE)	AP (Consistência Eventual)
Contador Plays	Chave-Valor	Redis	String (Operação atômica INCR)	AP (Sem Locks)
Histórico / Logs	Wide-Column	Cassandra	PK: <code>usuario_id</code> , CK: <code>reproduzido_em</code> DESC	AP (Escrita Peer-to-Peer)
Painel Artista	Wide-Column	Cassandra	PK: <code>artista_id</code> , CK: <code>reproduzido_em</code> DESC	AP (Query-First / Desnormalizado)

■ 5. Referências Bibliográficas

SADALAGE, P. J.; FOWLER, M. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley, 2013.

DeCANDIA, G. et al. Dynamo: Amazon's Highly Available Key-Value Store. SOSP, 2007.

CHANG, F. et al. Bigtable: A Distributed Storage System for Structured Data. OSDI, 2006.

LAKSHMAN, A.; MALIK, P. Cassandra: A Decentralized Structured Storage System.

LADIS, 2010.

REDIS DOCUMENTATION. Redis Data Structures & Cluster Architecture. Disponível em: <<https://redis.io/docs/>>. Acesso em: 2026.